

Прилог

Генерисање ChArUco табле

```
import numpy as np
import cv2, PIL, os
from cv2 import aruco
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd

dict = cv2.aruco.DICT_6X6_250 # Dictionary ID
v = 7 # Number of squares vertically
h = 5 # Number of squares horizontally
a_mm = 40 # Square side length (in mm)
m_mm = 30 # ArUco marker side length (in mm)
mv_mm = 8.5 # Margins size, vertical (in mm)
mh_mm = 5.0 # Margins size, horizontal (in mm)
print_margin_mm = 5.0 # printer limitation (in mm), it shortens each image by this amount at all borders

pixel_per_mm = 600.0 / 25.4
a = round(a_mm * pixel_per_mm)
m = round(m_mm * pixel_per_mm)
mv = round((mv_mm - print_margin_mm) * pixel_per_mm)
mh = round((mh_mm - print_margin_mm) * pixel_per_mm)

IMG_SIZE = (h * a, v * a)
OUTPUT_NAME = "ChArUco_A4_7x5_40A_30M.png"

def generate_board():
    dictionary = cv2.aruco.getPredefinedDictionary(dict)
    board = cv2.aruco.CharucoBoard(
        (h, v),
        a,
        m,
        dictionary,
    )
    img = cv2.aruco.CharucoBoard.generateImage(board, IMG_SIZE)
    image = cv2.copyMakeBorder(
        img, mv, mv, mh, mh, cv2.BORDER_CONSTANT, value=(255, 255, 255)
    )
    height, width = image.shape
    confirmed_image_size = (height, width)
    print(str(IMG_SIZE) + " resized to " + str(confirmed_image_size))
    # additional margin for printing, empirical
    mh_print = 50
    mv_print = round(mh_print * 297.0 / 210.0)
    print_image = cv2.copyMakeBorder(
        image, mv, mv, mh, mh, cv2.BORDER_CONSTANT, value=(255, 255, 255)
    )
    cv2.imwrite(OUTPUT_NAME, print_image)

generate_board()
```

Изрѳунавање параметара камере

```
import numpy as np
import cv2, PIL, os
from cv2 import aruco
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
import json

dict = cv2.aruco.DICT_6X6_250
v = 7
h = 5
a_mm = 40
m_mm = 30

pixel_per_mm = 600.0 / 25.4
a = round(a_mm * pixel_per_mm)
m = round(m_mm * pixel_per_mm)

directory = "calibration-photos"

def get_calibration_parameters(img_dir):
    # Define the aruco dictionary, charuco board and detector
    dictionary = cv2.aruco.getPredefinedDictionary(dict)
    board = cv2.aruco.CharucoBoard((h, v), a, m, dictionary)
    params = cv2.aruco.DetectorParameters()
    detector = cv2.aruco.ArucoDetector(dictionary, params)

    # Load images from directory
    image_files = [
        os.path.join(img_dir, f) for f in os.listdir(img_dir) if f.endswith(".jpg")
    ]
    all_charuco_ids = []
    all_charuco_corners = []

    # Loop over images and extraction of corners
    for image_file in image_files:
        image = cv2.imread(image_file)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        imgSize = image.shape
        image_copy = image.copy()
        marker_corners, marker_ids, rejectedCandidates = detector.detectMarkers(image)

        if len(marker_ids) > 0: # If at least one marker is detected
            print("len(marker_ids)" + str(len(marker_ids)))
            # cv2.aruco.drawDetectedMarkers(image_copy, marker_corners, marker_ids)
            ret, charucoCorners, charucoIds = cv2.aruco.interpolateCornersCharuco(
                marker_corners, marker_ids, image, board
            )

            if charucoIds is not None and len(charucoCorners) > 3:
                all_charuco_corners.append(charucoCorners)
                all_charuco_ids.append(charucoIds)

    # Calibrate camera with extracted information
    result, mtx, dist, rvecs, tvecs = cv2.aruco.calibrateCameraCharuco(
        all_charuco_corners, all_charuco_ids, board, imgSize, None, None
    )
    return mtx, dist

OUTPUT_JSON = "calibration.json"

mtx, dist = get_calibration_parameters(directory)
data = {"mtx": mtx.tolist(), "dist": dist.tolist()}

with open(OUTPUT_JSON, "w") as json_file:
```

```

json.dump(data, json_file, indent=4)

print(f'Data has been saved to {OUTPUT_JSON}')

```

Издавање одометрије из свих rosbag датотека

```

from bagpy import bagreader
import pandas as pd
import math
import os
from pathlib import Path

def get_yaw_from_quaternion(x, y, z, w):
    siny_cosp = 2.0 * (w * z + x * y)
    cosy_cosp = 1.0 - 2.0 * (y * y + z * z)
    return math.atan2(siny_cosp, cosy_cosp)

rootdir = "../bags"
extensions = ".bag"
for subdir, dirs, files in os.walk(rootdir):
    for file in files:
        ext = os.path.splitext(file)[-1].lower()
        if ext in extensions:
            print(os.path.join(subdir, file))
            b = bagreader(f'../bags/{file}')

            csvfiles = []
            odom_msgs = b.message_by_topic("/ros381/odom")
            csvfiles.append(odom_msgs)

            data = pd.read_csv(csvfiles[0])
            print(data)

            result = pd.DataFrame(
                {
                    "timestamp": data["Time"],
                    "x": data["pose.pose.position.x"],
                    "y": data["pose.pose.position.y"],
                    "phi": data.apply(
                        lambda row: get_yaw_from_quaternion(
                            row["pose.pose.orientation.x"],
                            row["pose.pose.orientation.y"],
                            row["pose.pose.orientation.z"],
                            row["pose.pose.orientation.w"],
                        ),
                        axis=1,
                    ),
                    "v": data["twist.twist.linear.x"],
                    "w": data["twist.twist.angular.z"],
                }
            )

            print(result)
            bag = Path(file).stem
            result.to_csv(f'{bag}.csv', index=False)
            print(f'Exported {len(result)} messages to {bag}.csv')

```

Конверзија ROS 2 bag датотека

```

rosbags-convert straight_test_slow

```

Контејнерско окружење

FROM docker.io/ubuntu:20.04

Set environment variables to avoid interactive prompts during installation
ENV DEBIAN_FRONTEND=noninteractive

Create ubuntu user
RUN useradd -m -s /bin/bash ubuntu

Update package lists and install basic utilities
RUN apt-get update && apt-get upgrade -y && \
apt-get install -y \
curl \
wget \
git \
vim \
nano \
htop \
net-tools \
build-essential \
software-properties-common \
apt-utils \
sudo \
&& apt-get clean \
&& rm -rf /var/lib/apt/lists/*

Give ubuntu user passwordless sudo
RUN echo "ubuntu ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers.d/ubuntu && \
chmod 0440 /etc/sudoers.d/ubuntu

Create mount point for workspace and set proper ownership
RUN mkdir -p /workspace && \
chown -R ubuntu:ubuntu /workspace

Set working directory
WORKDIR /workspace

Switch to ubuntu user
USER ubuntu

Set locale
RUN sudo apt-get update && \
sudo apt-get install -y locales && \
sudo locale-gen en_US.UTF-8 && \
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

RUN sudo apt-get update && \
sudo apt-get install -y \
neovim \
ranger \
&& sudo apt-get clean

X11 and GUI libraries
RUN sudo apt-get update && \
sudo apt-get install -y \
x11-apps \
xauth \
libx11-6 \
libxext6 \
libxrender1 \
libxtst6 \
libxi6 \
libxkbcommon-x11-0 \
libgl1-mesa-dri \
libglx-mesa0 \
mesa-utils \
dbus-x11 \
&& sudo apt-get clean

```

# GTK support for GUI apps
RUN sudo apt-get update && \
  sudo apt-get install -y \
  libgtk-3-0 \
  libgtk-3-bin \
  libcanberra-gtk3-module \
  && sudo apt-get clean

# Qt support for GUI apps
RUN sudo apt-get update && \
  sudo apt-get install -y \
  libqt5core5a \
  libqt5gui5 \
  libqt5widgets5 \
  qt5-gtk-platformtheme \
  && sudo apt-get clean

# Python GUI dependencies
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-tk \
  python3-pyqt5 \
  && sudo apt-get clean

# SVO specific
RUN sudo apt install -y \
  libboost-all-dev \
  libeigen3-dev
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-opencv \
  libopencv-dev \
  && sudo apt-get clean

# ROS 1 Noetic setup
RUN sudo DEBIAN_FRONTEND=noninteractive DEBCONF_NONINTERACTIVE_SEEN=true \
  apt-get install -y -o Dpkg::Pre-Install-Pkgs::=/bin/true \
  keyboard-configuration && \
  sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com --recv-keys F42ED6FBAB17C654 || true && \
  sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu focal main" > /etc/apt/sources.list.d/ros-latest.list' && \
  sudo apt-get update && \
  sudo DEBIAN_FRONTEND=noninteractive apt-get install -y --allow-unauthenticated ros-noetic-desktop-full && \
  sudo apt-get clean

# Additional ROS build tools and libraries
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-catkin-tools \
  python3-vcstool \
  python3-osrf-pycommon \
  libglew-dev \
  libopencv-dev \
  libyaml-cpp-dev \
  libblas-dev \
  liblapack-dev \
  libsuitesparse-dev \
  && sudo apt-get clean

# Source ROS setup in bashrc
RUN echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc

```